4. Data Schema

Two changes were needed: a new table to hold bookings, and two columns on events so that a weather forecast can be requested for the right place. Everything else from Assessment 2 was left alone, because it already modelled the case study correctly.

Table 2 - The registrations table.

Column	Type	Why it exists
registration_id	INT AUTO_INCREMENT PK	Surrogate key. The natural key would be (event_id, email), which is two wide columns; a single integer is cheaper to index and easier to reference.
event_id	INT NOT NULL, FK -> events	Which event was booked. ON UPDATE CASCADE, ON DELETE RESTRICT.
first_name / last_name	VARCHAR(80) NOT NULL	Stored separately from email so the staff screen can sort and greet by name, and so a shared family address still reads correctly.
email	VARCHAR(150) NOT NULL	Contact detail, and part of the uniqueness rule.
phone	VARCHAR(30) NOT NULL	The charities ring people when a event is cancelled - an email sent the night before is often missed.
tickets	INT NOT NULL DEFAULT 1	How many places were bought. CHECK (tickets > 0 AND tickets <= 50).
registration_date	DATE NOT NULL	The day the visitor says they booked. This is a business value, not a timestamp, which is why it is a DATE.
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	When the row was actually written. Kept separate from registration_date so the two can never be confused when auditing.
notes	VARCHAR(255) NULL	Dietary requirements, accessibility needs, 'arriving late'.

The constraints are where the design actually lives, so they are worth spelling out:

```
CONSTRAINT fk_registration_event
    FOREIGN KEY (event_id) REFERENCES events (event_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,

CONSTRAINT uq_registration_event_email
    UNIQUE (event_id, email),

CONSTRAINT chk_tickets_positive CHECK (tickets > 0),
CONSTRAINT chk_tickets_sane     CHECK (tickets <= 50),
CONSTRAINT chk_email_shape      CHECK (email LIKE '%_@%_._%')
```

The UNIQUE key is the answer to the first problem in Section 2: 'one person, one booking per event' becomes a property of the table, so it holds no matter how many requests arrive at once, and the API only has to translate the resulting duplicate-key error into a friendly 409. ON DELETE RESTRICT is the answer to the second problem: MySQL itself refuses to delete an event that still has bookings, so the rule